Evolutionary Algorithms: Final report

Carolina Alba Marugan Rubio (r1072969)

December 28, 2025

1 Metadata

- **Group members during group phase:** Adrien Loiseau, Carolina Marugan Rubio, Merlijn Pietercil and Aikin Verhoeven
- **Time spent on group phase:** 10 hours
- **Time spent on final code:** 40-45 hours
- **Time spent on final report:** 20 hours

2 Changes since the group phase (target: 0.25 pages)

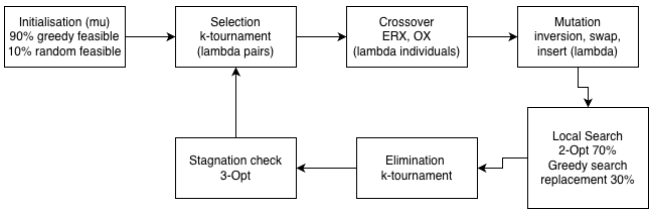
1. I created a visualisation feature for diversity promotion (count of unique edges).
2. After some experiments explained with more detail in Section 3.4, I changed the initialisation phase by modifying some parameters.
3. I spent a lot of time reorganising the genAI code from the group phase since it did not respect clean code good practices, so I cleaned it and better organised it in functions similarly to how we did it during the programming sessions.
4. I implemented a Numba version of the methods listed in Section 3.13
5. I performed experiments with four different crossover operators (Edge Recombination, Order Crossover, Cycle Crossover, Partially Matched), but ended up choosing only two of them (ERX and OX). ERX was not developed during the group phase (section 3.7)
6. I added a stagnation solution using 3-Opt for a portion of the offsprings.
7. I changed simple elimination of worst candidates during evaluation for k-tournament evaluation during elimination.
8. I added a hybrid Local Search Operator (LSO) that balances 2-Opt with a greedy search replacement (section 3.9)

3 Final design of the evolutionary algorithm (target: 3.5 pages)

3.1 The three main features

1. Greedy but feasible initialisation (Section3.4).
2. Hybrid Local Search strategy (Section 3.9).
3. Feasibility-Aware Initialization and Operators (mutation, crossover).

3.2 The main loop



3.3 Representation

Solutions are encoded as 1D NumPy arrays containing a permutation of city identifiers (path representation). The array index corresponds to the visit step; for example, $[0, 2, 3, 1]$ signifies a path visiting cities 0, 2, 3, and 1 sequentially. The population and offspring are a list[np.ndarray].

The motivation is the resulting efficiency when computing the cost of each path or tour, a highly used computation throughout the algorithm loop. An input matrix representing the distances between all nodes, and this representation, allows for direct mapping of indexes when looping the tour array.

I considered an alternative representation mapping cities to their visit positions (where cycle[city] = order). This was discarded because fitness evaluation (highly used) becomes computationally expensive.

3.4 Initialization

During the group phase, our code initialised the entire seed population using a randomised greedy heuristic that worked as follows:

- **Pre-computation**: For every city, a list of feasible outgoing neighbors is pre-calculated and sorted by distance (ascending).
- **Construction**: Starting from a random city, the tour is built step-by-step. At each step, instead of deterministically picking the single nearest unvisited neighbor, the algorithm selects the next city randomly from the nearest k feasible neighbors.
- **Feasibility Check**: If the construction is infeasible, the algorithm attempts a simple 2-swap repair near the end of the tour. If repair fails, it restarts the process until the retries variables allow it.

Although the heuristic combines a greedy strategy with a degree of randomness that promotes diversity, my understanding from the lectures and from the Eggholder problem was that applying a strong local search at initialization can lead to early stagnation. This behavior appeared with *tour50*, where the algorithm converged very quickly without further improvement. To mitigate this, I initially chose to randomly initialize 80% of the population with feasible tours and to apply the heuristic only to the remaining 20

However, when running the algorithm with *tour1000*, I observed that, given the high computational cost of the operators and the limited time window, the algorithm required a strong push from the beginning; otherwise, it was impossible to outperform the heuristic itself. The key insight was that this heuristic does not refine existing tours toward the same local optimum, as in continuous local search, but instead optimizes each tour independently from a random starting point. This allows different local optima in the discrete search space to coexist within the initial population. As a result, I shifted the initialization strategy to a 90/10% greedyrandom split in favor of the heuristic.

Since the heuristic already incorporates diversity by randomly selecting among the top $k = 5$ neighbors, I reduced k to 1 in order to obtain better cheaper paths. Additionally, I realized that the heuristics inherent randomness, combined with the large problem size (*tour1000*), required a larger population to achieve a reasonable probability of generating high-quality feasible tours. For this reason, I increased the population size to 500. After these changes, the algorithm was able to outperform the heuristic.

3.5 Selection operators

Parent selection is performed using k-tournament selection. Two distinct tournaments are conducted to select two parents. An explicit check ensures that the two selected parents are distinct individuals. I chose $k = 5$ because I wanted to add a small selective pressure, since I am already introducing a lot of randomness in other parts of the loop. I chose this operator because it is the one that balances exploitation with exploration best.

At the beginning of the project, I also considered using ranking-based selection with linear or exponential decay. However, I decided not to implement a decay scheme without first having a reliable diversity-preservation mechanism in place. With decay, especially towards the final iterations, the probability of selecting the best individuals becomes very high, leading to excessive selective pressure and favoring stagnation, which already appeared to be present in some configurations (e.g., with $tour50$). To counteract this effect, I aimed to introduce explicit diversity mechanisms, but since these could not be successfully implemented at the time, ranking-based selection with decay was ultimately not explored.

3.6 Mutation operators

To balance exploration and exploitation, the algorithm implements three distinct mutation operators: Inversion, Swap, and Insert (described in [1]). The Scramble operator was excluded, as its effects are effectively mimicked by the collective action of the other three. I believe there is "No Free Lunch" for these operators. If you apply inversion, it is possible to speed up the search and find a better candidate; however, using it all the time can also 'break' a good candidate that was hard to obtain, when a simple swap would have worked better. To mitigate these trade-offs, my algorithm applies one operator per mutation event, selected via a uniform distribution. This strategy ensures a dynamic balance between structural variation and local refinement, with the overall degree of randomness remaining controllable through the global mutation rate.

3.7 Recombination operators

To maximize the inheritance of beneficial structural traits, the algorithm employs two complementary operators: Edge Recombination Crossover (ERX) and Order Crossover (OX). ERX was chosen due to its superior ability to preserve edges from both parents [2]. However, the computational complexity of generating edge maps initially rendered ERX prohibitively slow; it was only successfully integrated after implementing Numba optimizations to ensure feasible runtime. OX was selected as a robust alternative that preserves the relative order of subsequences.

To maintain diversity, the algorithm selects between ERX and OX using a uniform distribution at each recombination step. Alternative operators were considered but discarded: Partially Mapped Crossover (PMX) was excluded as it preserves adjacency information less effectively [1].

3.8 Elimination operators

The algorithm employs a $(\mu + \lambda)$ survivor selection using k-tournament selection. First, duplicates are removed from the combined pool. Then, in order to maintain population diversity and prevent premature convergence, elitism is applied to retain the single best individual. The remaining population slots are filled by repeatedly identifying winners of random tournaments of size $k = 3$ from the candidate pool. I also tried a $(\mu + \lambda)$ survivor selection scheme. A strict duplicate elimination mechanism was applied, ensuring that only unique individuals were considered for the next generation. This guarantees that the best solution found so far survives (elitism). Nevertheless, k-tournament seemed to give slightly better results.

3.9 Local search operators

- **Steepest Descent 2-Opt (First Improvement and Hybrid)**:
  - Probability: 70%
  - Mechanism: It iterates through all possible edge-pair swaps. If a swap improves the tour, it applies it immediately and restarts the inner scan. It continues until no more improvements can be found or an evaluation budget is exhausted (defined by parameters). But it only applies this to 40% of the offsprings.
- **Greedy Search Replacement (Reconstruction)**:
  - Probability: 30%
  - Mechanism: It takes the worst 20% of the offspring and completely rebuilds them. It keeps the single best edge from the original tour as a "seed", effectively destroying the rest of the tour, and then rebuilds the entire path using a pure greedy heuristic (always picking the nearest unvisited neighbor).
- **Additionally (Conditional): 3-Opt**
  - Condition: Only applied if the global best fitness stagnates for a stagnation limit (10) generations.

Additionally, most of the operators try to only produce feasible tours, which could be considered small LSOs. These LSOs caused a more significant improvement as the size of the tour increased. They were essential for $tour1000$.

3.10 Diversity promotion mechanisms

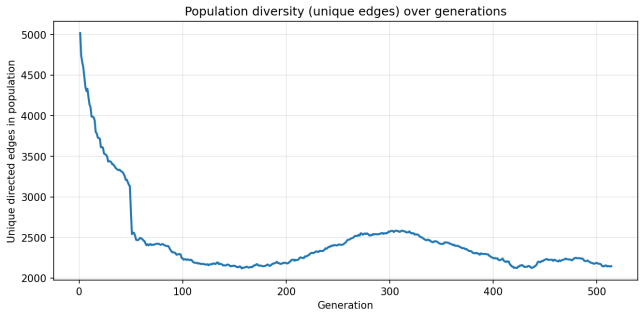
This was one of the hardest parts of the project, as trying to quantify distances between permutations with low operation complexity was not trivial. There was a trade-off algorithm speed versus diversity promotion. An explanation of the mechanism to track and visualise diversity in the algorithm is explained in Figure 2.

Diversity is promoted primarily through a strict duplicate elimination mechanism during survivor selection, which prevents the proliferation of clones. Additionally, diversity is injected via seeded random initialization (10% of the initial population) and maintained dynamically by a destructive Greedy Search Replacement operator that reconstructs the worst individuals in each generation (applied to the worst 20% of individuals with a 30% probability).

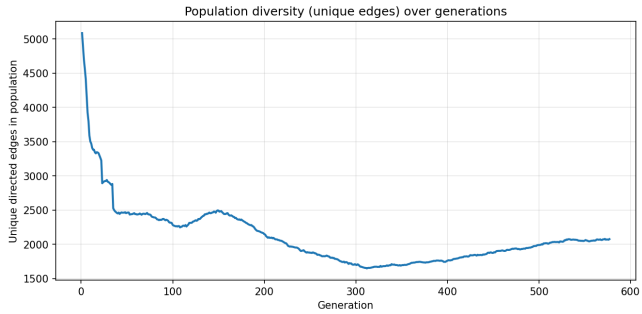
As we can see in Figure 2, there is a visible improvement in the diversity of the population.

3.11 Stopping Criterion

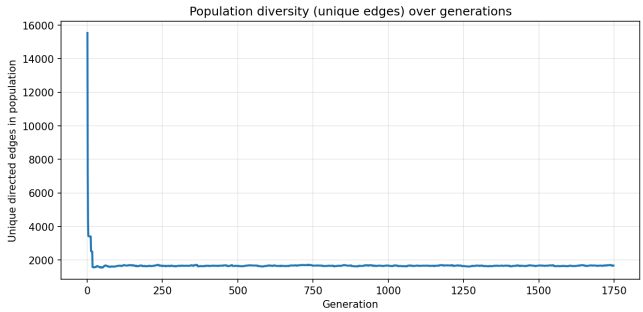
Integrating LSO and ERX significantly increased computational costs for larger tours, necessitating Numba optimizations. Consequently, as the algorithm exhibited continuous improvement without stagnation, a convergence-based stopping criterion was deemed unnecessary. A simple stagnation detection rule (e.g., terminating after 200 non-improving iterations) would have sufficed only for the *tour50* instance.



(a) Diversity evolution when applying 3-opt when stagnating after 5 generations



(b) Diversity evolution when applying 3-opt when stagnating after 15 generations



(c) Diversity evolution right after the group phase

Figure 2: *tour*1000 Diversity promotion to overcome stagnation versus no diversity promotion. The used technique is counting for each generation, the number of unique edges in the population.

3.12 Parameter selection

I did not have time to use any automatisation for parameter tuning. I tried to reason how much I should vary each parameter based on the evolution of the mean and best fitness, the speed of convergence, the diversity promotion visualisation feature, and trial and error.

3.13 Other considerations

I optimised the computational time by using Numba and adapting the following operations to Numba methods: all the crossover and mutation operators, 2 and 3 Opt search, fitness evaluation, greedy search replacement, and the tournament selection comparison logic.

4 Numerical experiments (target: 1.5 pages)

4.1 Metadata

Table 1: Parameters for the EA

Parameter name	**Value**
μ	500
λ	500
k-tournament	5
mutation rate	0.8
crossover rate	0.8
greedy initialisation tries	10000
feasible variation retries	20
greedy search replacement probability	0.3

After many experiments I thought that changing some parameters like population and offspring size can make a big difference for any type of input. I still think this, specially for the case of *tour*1000 in the initialisation phase. But eventually I also realised that part of this difference can also be due to randomness. I was going to vary the population size depending on the input so that less computations were needed for some tour sizes and therefore more iterations to find a better solution were possible. Nevertheless, I still opted for choosing a less risky population size of 500.

The EA was executed on a MacBook Air (Late 2020) equipped with the Apple M1 System on Chip (SoC). This processor features an 8-core CPU architecture (comprising four high-performance cores and four high-efficiency cores) with a maximum clock frequency of approximately 3.2 GHz. The system is configured with [8 GB / 16 GB] of Unified Memory, and all simulations were performed using Python version 3.12.2.

4.2 tour50.csv

The best tour length found was 13844.338. The best cycle is: [40,11,10,38,8,22,6,19,23,49,2,32,15,43,45,26,20,47,44,17, 12,29,13,31,27,25,16,42,18,21,48,5,35,41,30,7,9,14,37,33,34,28,39,4,46,0,36,24,3,1]. The Figure 3 was taken from the previous development phase of the EA, since it was already able to find the same result. In comparison to Figure 5, we see that the mean is much more stable in Figure 3 because it does not have diversity promotion yet. We can see that it converges very quickly and stagnates for the remaining computation, possibly really close to the optimal solution.

In Figure 4 we see the results for running the algorithm 500 times and comparing the variation in the best and mean results. The algorithm always finds the same best result, while there is variability in the final mean results.

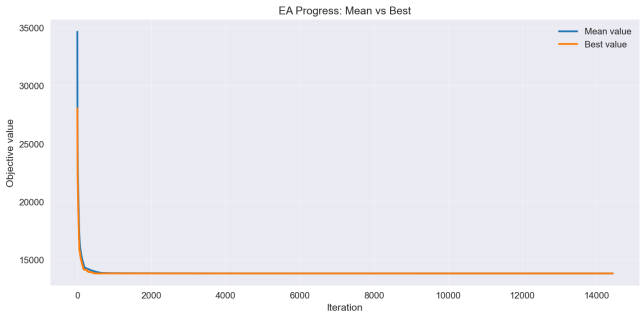


Figure 3: EA progress (Mean vs Best) $tour50$

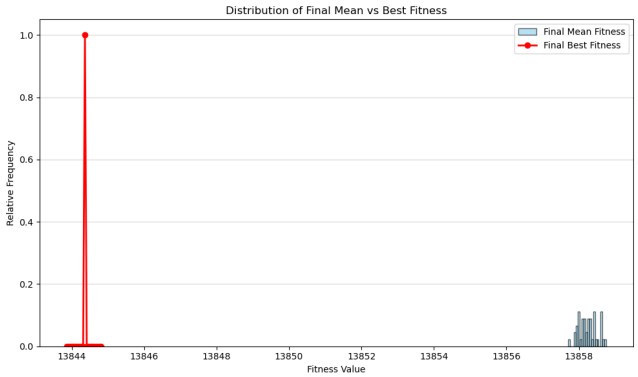


Figure 4: Histogram of the final mean fitnesses and the final best fitnesses of the 500 run experiment

4.3 tour250.csv

The best tour length found was 75104.822472. Figure 5 represents the evolution of the mean and best value for each generation. This figure was already taken when the diversity mechanisms were implemented (described in Section 3.10). We can see how much the mean values vary thanks to the diversity.

4.4 tour1000.csv

In Figure 6 we see the evolution of $tour1000$ with the best result as 69383. Figure 6 was also taken when diversity mechanisms were not implemented, but still the algorithm managed to get a similar best solution, only after applying the LSO in the initialisation.

5 Critical reflection **(target: 1.5 page)**

5.1 Strengths and weaknesses

The three main strengths of evolutionary algorithms in my experience are as follows:

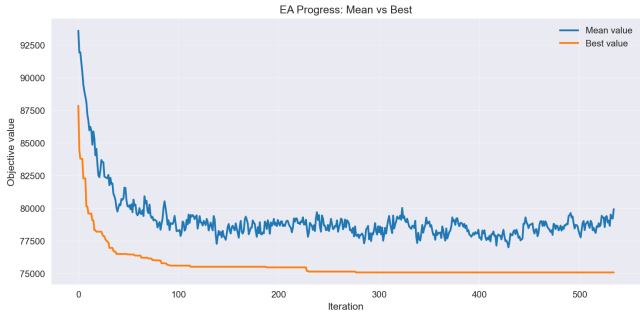


Figure 5: EA progress (Mean vs Best) $tour250$

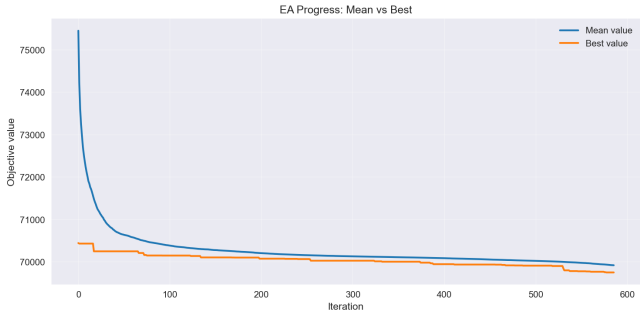


Figure 6: EA progress (Mean vs Best) for $tour1000$

1. **Reasonably good results in less time**: according to the No Free Lunch theorem [1], there is no algorithm that outperforms the rest in all types of problems. However, Evolutionary Algorithms (EAs) can find a solution that is close to the global best in less time than exact mathematical methods like Branch and Bound, specifically in NP-Hard problems like the one developed in this project (Travelling Salesperson Problem). This makes EAs an excellent choice for complex real life problems when there is a time or resource constraint.
2. **Less likely to get trapped in a local optimum**: compared to gradient based optimisation methods, and thanks to the diversity promotion, randomisation features, and the exploration-exploitation balance (if developed correctly), EAs are less prone to get stuck in a local optima.
3. **Domain-independent**: evolutionary algorithms do not require the objective function to be continuous, differentiable, or even mathematically defined (unlike gradient-based or SIMPLEX methods). This means they can be applied to a wider domain of fields.

The three main weak points of evolutionary algorithms in my experience are as follows:

1. **High computational cost for initialisation, evaluation and local search operators**: EAs are population-based and they require evaluating the fitness of hundreds or thousands of individuals over many iterations. Additionally, they can require the use of heuristics that are computationally expensive.
2. **Sensitivity to parameter tuning and Data dependence**: EAs can perform differently for different data and for different sizes of data, which means that they need to be tailored in accordance, and reuse of hyperparameters does not always lead to the best result.
3. **Risk of stagnation**: if there is no balance between exploration and exploitation by introducing the right amount of randomness and diversity, EAs are prone to premature convergence.

One of the things that surprised me the most during the development of this project is how hard it is to tune all the parameters, because even the ones that might seem insignificant, like the k I used in my initialisation heuristic, or the maximum number of retries for every method that returns only feasible paths, can make a big difference when slightly changed. Even though I already knew that EAs were data dependant, with this practice I became much more aware of how much a result can change depending on the size of the data and the chosen parameters.

EAs are highly appropriate for the TSP when the goal is to find reasonably good solutions in a practical amount of time. Specially for real-world industry problems that necessitate complex constraints. While EAs

suppose significant effort in terms of representation design and hyperparameter tuning, their ability to escape local optima and integrate local search makes them a robust alternative to exact methods that may struggle with the combinatorial explosion of the search space.

5.2 GenAI peer reviewing critical reflection

The weaknesses of the AI-generated peer review compared to the human-written one are, in my opinion, the tendency to be either too general and vague or overly specific in a way that misses the big picture. In particular, it struggles to both identify specific qualities but still explicitly counterbalance them with broader considerations that are expected in a thorough review. For example, the GenAI correctly noted that the operator chosen by the other group was very good; however, it did not sufficiently acknowledge that many other aspects must also be considered when evaluating the performance of a specific operator, and so it did not contrast this positive point with potential limitations.

Did Copilot identify issues the humans missed, or vice versa is difficult to determine in this case, since only two issues were required. One identified issue overlapped between the human and AI reviews, while the other differed, which does not imply that either is incorrect. Both the GenAI and the human reviewers successfully pinpointed two relevant issues. Copilots review was useful, as it provided good ideas accompanied by reasonably clear explanations.

While the AI-generated review can serve as a useful introduction (assuming it is correct), actively reviewing the work leads to a much deeper understanding. In what ways can AI be a helpful tool in peer review is by introducing new ideas when it has more background knowledge than the reviewer, while in what ways can it be harmful is that it may also have less knowledge, state incorrect facts, or lack a mature and nuanced view of the topic.

My view towards reviewing reports using generative AI is that the best strategy is to combine both approaches whenever possible: GenAI can offer new ideas or alternative perspectives that humans might not consider, but human supervision, double-checking, and the ability to balance detailed analysis with a broader view remain essential. I would clearly prefer to be reviewed by a human. A broader perspective, which Large Language Models might lack, also involves empathy, which most humans naturally have and which GenAI can only emulate when explicitly instructed.

How might relying too much on AI affect our ability to critically evaluate scientific work in the future relates mainly to attention to detail, as excessive reliance on AI may encourage faster but more superficial analysis; this may be acceptable for simpler tasks but can hinder the development of more sophisticated and innovative solutions for complex problems.

5.3 GenAI critical reflection

Generative AI helped me during the project in the following ways:

1. **Efficiency and learning depth under time constraints**: given the limited time available, using GenAI to assist with code generation allowed me to save a significant amount of time. As a consequence, I was able to dedicate more effort to properly understanding how each technique works, including their low-level mechanisms, and to reason more confidently about which methods are better suited for which problems, what would happen under alternative assumptions, and how to interpret the results.
2. **Accelerating the search for information**: it streamlined internet searches and sometimes surfaced relevant concepts that I would have only found by navigating many different sources. This increased speed made it possible to learn new material that was not covered in class.

I did you not observe any or sufficient added value for the following:

1. **Code quality and incorrect information**: If one aims to truly understand what the algorithm is doing, it becomes necessary to carefully analyze all the generated code, which may include unnecessary, poorly implemented, or confusing components. This can result in spending a significant amount of time understanding code that later needs substantial modification or even a complete rewrite, especially when the proposed class structure is questionable (for example, when some attributes should not be class-level). In such cases, it is necessary to weigh the pros and cons of the structure suggested by GenAI, which is often presented in a disorganized way and with poor coding practices. If the user is not already familiar with good coding practices, there is also a risk of learning incorrect patterns. LLMs can hallucinate and provide incorrect information even for non-technical queries. Without verification, this can lead to adopting false assumptions or facts.
2. **Problem understanding**: generative AI can know how to program a specific operator but still fail to properly assess whether it can be applied to a given problem, or at which stage of the problem it is appropriate

to use it; this strongly depends on the LLM, as some handle this aspect better than others. I became aware of this limitation even before starting the project, while experimenting with the Eggholder function.

6 Generative AI compliance form (does not count toward page limit)

6.1 Recombination operator

Code skeleton generation for a recombination operator of permutations.

Generative AI model: Microsoft Copilot

Motivation: It is a well known operator, not a new idea. The intention was to save time, since I already knew how the operator worked.

Methodology:

> **Prompt:**
> Do you know ERX crossover operator? can you give me the code for it? The header of the method should be: def _crossover_ERX(self, p1: np.ndarray, p2: np.ndarray) -> np.ndarray:

Postprocessing: the generated code was usually correct. In cases of occasional errors, I corrected them by hand.

Reflection: since I asked for a small block of code, it was likely that the code was correct. I did not learn that much from this because I already knew how the operator worked. The code was very useful.

6.2 Method adaptation to Numba

This prompt was asked several times for most of the methods with the purpose of trying to speed up most of them.

Generative AI model: Microsoft Copilot

Motivation: It was a complex task using a library that I did not previously know.

Methodology:

> **Prompt:**
> Can you convert this method to a numba version?
> (pasted method)

Postprocessing: the generated code was usually correct. In cases of occasional errors, I corrected them by hand or I asked the genAI for help solving them.

Reflection: I learned that numba uses annotations, which can be something interesting to further explore in the future.

6.3 Assisted help for coding LSOs and diversity promotion mechanism

Code skeleton generation for a LSO and a diversity promotion mechanism. A very similar prompt was given for diversity promotion, where I give the LLM my idea of implementation and give limited indications about the header of the method.

Generative AI model: Microsoft Copilot

Motivation: the implementation of this heuristic is very time consuming

Methodology:

> **Prompt:**
> Can you help me code this heuristic for a LSO in my EA? The context is that I am using a path representation for the TSP. I want to (here I wrote a similar explanation to the one I gave in 3.9 and I gave indications of what the input and output should be).

Postprocessing: the generated code was usually correct. In cases of occasional errors, I corrected them by hand or I asked the genAI for help solving them. I also modified small portions of code that might not be necessary.

Reflection: The LLM provided nice recovery mechanisms that I didn't ask for when making the tours feasible.

References

[1] A. E. Eiben and J. E. Smith. *Introduction to Evolutionary Computing*. Springer, 2 edition, 2015.

[2] Pedro Larrañaga, C. M. H. Kuijpers, R. H. Murga, I. Inza, and S. Dizdarevic. Genetic algorithms for the Travelling Salesman Problem: A review of representations and operators. *Artificial Intelligence Review*, 13(2):129–170, 1999.